

Laporan Analisis Sistem Distribusi Pesan & File Berbasis Hybrid Socket

****Mata Kuliah:**** Jaringan Komputer Lanjut

****Topik:**** TCP multi-client, UDP pinger/heartbeat, message boundary, skalabilitas socket, QUIC, dan bind eksplisit pada client

Bagian A — Implementasi Pemrograman Soket

A1. Layanan TCP Multi-Client

Welcoming socket dibuat sekali di server, lalu dipakai untuk `listen()` dan `accept()`. Setiap hasil `accept()` menghasilkan connection socket khusus untuk satu client, kemudian server membuat thread baru.

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
server_socket.bind((args.host, args.port))
```

```
server_socket.listen(100)
```

```
while True:
```

```
    connection_socket, client_address = server_socket.accept()
    thread = threading.Thread(
        target=handle_client,
        args=(connection_socket, client_address, next_client_id),
        daemon=True
    )
    thread.start()
    next_client_id += 1
```

****Perbedaan lifecycle `serverSocket` dan `connectionSocket`:****

- `serverSocket` adalah welcoming socket. Hidup sejak server start sampai server dimatikan. Tidak dipakai mengirim/menerima data aplikasi.

- `connectionSocket` dibuat oleh `accept()` untuk satu client. Hidup selama sesi client tersebut. Setelah client selesai, socket ditutup dan resource dilepaskan.

A2. Layanan UDP Pinger

Client mengirim 10 pesan PING dan menggunakan timeout 1 detik agar paket yang hilang tidak membuat program menunggu selamanya.

```
client_socket.settimeout(1.0)
```

```
for seq in range(1, 11):
```

```
    send_time = time.time()
    client_socket.sendto(f'PING {seq} {send_time}'.encode('utf-8'), server_address)
    try:
        data, _ = client_socket.recvfrom(65535)
        rtt_ms = (time.time() - send_time) * 1000
        rtt_ms.append(rtt_ms)
        print(f'#{seq:02d} RTT={rtt_ms:.3f} ms')
    except socket.timeout:
        lost += 1
        print(f'#{seq:02d} timeout')
```

RTT dihitung sebagai `waktu diterima - waktu dikirim`, lalu diambil minimum, maksimum, dan rata-rata. Packet loss dihitung dari jumlah timeout terhadap total pengiriman.

Bagian B — Analisis Mendalam & Arsitektur Jaringan

B1. Byte-Stream vs Message Boundary

TCP menggunakan `SOCK\_STREAM` dan menyediakan aliran byte yang reliable, ordered, dan connection-oriented. TCP tidak menyimpan konsep "satu pesan" sebagai unit aplikasi. Yang dijamin hanya urutan byte yang diterima sama dengan urutan byte yang dikirim. Karena itu TCP disebut **byte-stream channel** tanpa message boundaries

UDP menggunakan `SOCK\_DGRAM`. Setiap pemanggilan `sendto()` menghasilkan satu datagram yang dikirim sebagai unit independen. Saat diterima dengan `recvfrom()`, satu pemanggilan `recvfrom()` umumnya menerima satu datagram utuh selama ukurannya cukup. Karena itu UDP mempertahankan **message boundaries**, meskipun tidak menjamin delivery, urutan, atau bebas duplikasi.

Dampak teknis bila client TCP mengirim 3 string berturut-turut tanpa delay:

Server dapat menerimanya sebagai satu potongan gabungan, terpotong, atau campuran parsial karena TCP hanya melihat byte stream. Misalnya server melakukan `recv(1024)` sekali dan ternyata menerima bagian dari pesan 1 + seluruh pesan 2 + sebagian pesan 3.

Rancangan protokol aplikasi pada tugas ini:

Setiap pesan dibungkus frame:

[4 byte panjang payload][payload UTF-8]

Penerima wajib membaca tepat 4 byte header, membaca panjang payload, lalu membaca payload sesuai panjang tersebut. Dengan cara ini batas pesan dipulihkan di layer aplikasi.

B2. Analisis Skalabilitas Socket

Jika server TCP melayani `N` client bersamaan dari IP berbeda, jumlah socket/file descriptor minimal adalah:

- 1 welcoming socket
- `N` connection socket
- bisa ditambah descriptor standar seperti stdin, stdout, stderr

Jadi total minimal untuk keperluan layanan TCP adalah **$N + 1$ socket**, belum termasuk file descriptor proses lain.

Server UDP hanya membutuhkan **1 socket** karena UDP tidak membuat koneksi per client. Socket UDP berfungsi sebagai endpoint `IP:port` yang menerima datagram dari banyak sumber. `recvfrom()` memberi tahu alamat pengirim, sehingga server dapat membalas ke alamat tersebut dengan `sendto()`.

Implikasinya:

- Konsumsi memori OS lebih kecil untuk UDP karena tidak ada state koneksi per client seperti TCP.
- UDP hanya perlu bind ke satu port, misal `12001`, dan tetap bisa melayani banyak client.
- Kelemahannya, aplikasi harus sendiri mengelola reliabilitas, urutan, retransmisi, dan sesi bila dibutuhkan.

B3. Evolusi Transport & QUIC

QUIC memilih berjalan di atas UDP karena UDP sudah didukung luas oleh sistem operasi, router middlebox, firewall, NAT, dan perangkat jaringan yang ada. Membuat transport protocol baru dari awal akan menghadapi masalah deployability karena banyak middlebox memblokir protokol transport non-standar. Menumpang penuh pada TCP juga tidak ideal karena TCP diproses di kernel OS, sehingga sulit diperbarui cepat dan sulit berevolusi.

Dengan berjalan di atas UDP, QUIC:

- dapat diimplementasikan di user-space,
- lebih cepat berevolusi,
- tetap kompatibel dengan infrastruktur yang ada,

- dapat membangun fitur modern seperti encrypted transport, connection migration, dan multiplexed stream sendiri.

QUIC menyelesaikan HOL blocking pada level transport dengan ****multiplexing banyak stream independen**** dalam satu koneksi. Jika satu paket hilang, hanya stream yang bergantung pada data tersebut yang terhambat; stream lain masih bisa dikirim dan diterima. TCP mengalami HOL blocking karena aliran byte tunggal harus tiba dengan urutan sempurna; satu segment yang hilang dapat menahan seluruh aliran, termasuk data dari request HTTP yang sebenarnya independen.

B4. Skenario Bind Eksplisit pada Client

Jika pada `UDPClient.py`` ditambahkan:

```
clientSocket.bind(("", 5432))
```

maka client tidak menggunakan ephemeral port acak, melainkan mengikat dirinya pada semua alamat lokal dengan port `5432``.

Server tetap bisa membalas pesan tersebut karena server mengetahui alamat sumber dari `recvfrom()`, yaitu `IP_client:5432``, lalu membalas ke alamat itu.

Kendala muncul jika dua instansi client berjalan pada host yang sama dan sama-sama melakukan bind ke port `5432``. Bind kedua akan gagal karena port tersebut sudah dipakai. Secara umum tanpa opsi khusus seperti `SO_REUSEADDR`/reuse-port`, dua socket tidak boleh mengikat endpoint lokal yang sama. Dampaknya:

- client kedua gagal start atau gagal bind,
- pengembangan/testing multi-client di satu host menjadi terbatas,
- manajemen port client menjadi lebih rentan konflik.

Bukti Wireshark yang Perlu Dilampirkan

Untuk melengkapi laporan maksimal 5 halaman, tangkap bukti berikut saat menjalankan program:

1. ****Filter TCP:**** `tcp.port == 12000``

Tunjukkan handshake `SYN``, `SYN-ACK``, `ACK``, lalu data aplikasi yang membawa frame TCP.

2. ****Filter UDP:**** `udp.port == 12001``

Tunjukkan 10 paket `PING`` dari client dan balasan server, termasuk paket yang hilang bila diuji dengan delay/hambatan jaringan.

3. Tambahkan analisis singkat mengenai perbedaan flag TCP dan tidak adanya handshake pada UDP.

Kesimpulan

Hybrid socket cocok untuk tugas ini karena TCP memberikan channel pesan reliable untuk komunikasi data, sedangkan UDP memberikan mekanisme ringan untuk heartbeat/pinger. Masalah utama TCP tanpa message boundary diatasi dengan protokol frame ber-header panjang tetap. UDP efisien untuk skenario request-reply sederhana, tetapi membutuhkan timeout dan perhitungan statistik untuk menangani paket hilang.